Bunny Carrot Maze Based on Q-Learning

Author: HUANG Yuting (5537515)

Github: https://github.com/UnicornYumi/Bunny-Carrot-Maze.git

Youtube: Show in Github

1 Introduction

This project applies the Q-Learning algorithm to design and implement a two-stage maze navigation game called Bunny Carrot Run. In this game, an agent-controlled bunny must first collect a carrot (key) and then reach the exit while avoiding traps and managing its health points (HP).

The project is implemented using Python stack. The tabular Q-Learning algorithm is implemented to enable the agent to learn action-value functions through temporal-difference updates and epsilon-greedy exploration. Pygame [1] is used as the game development framework, providing support for graphical rendering, event handling, dynamic updates, and user interface design.

2 Game Design

2.1 Rules of the Game

The game environment is a procedurally generated maze where a bunny is controlled by a reinforcement learning agent. The objective of the game is for the bunny to collect a carrot (key) and then reach the exit while avoiding hazards and managing its health points (HP). Each time the environment resets, a new valid maze layout is generated, and the carrot, exit, poison traps, and medkits are placed on accessible floor cells.

Core rules of the game are as follows:

1. The bunny can perform five discrete actions: UP, DOWN, LEFT, RIGHT, and STAY.
2. The bunny must collect the carrot before reaching the exit; otherwise, the game cannot be completed.
3. Stepping on a poison trap reduces HP and results in a negative reward.
4. Picking up a medkit restores HP, but the HP value cannot exceed the predefined maximum.
5. Attempting to move into a wall results in an immediate penalty.
6. The episode terminates under two conditions:
   - success: the bunny reaches the exit after collecting the carrot,
   - failure: the bunny's HP decreases to zero.

This game setup creates a structured two-stage objective (collect key and reach exit). The agent must learn to balance exploration, survival, and long-term goal achievement to maximize cumulative reward, workflow is shown in Fig1.

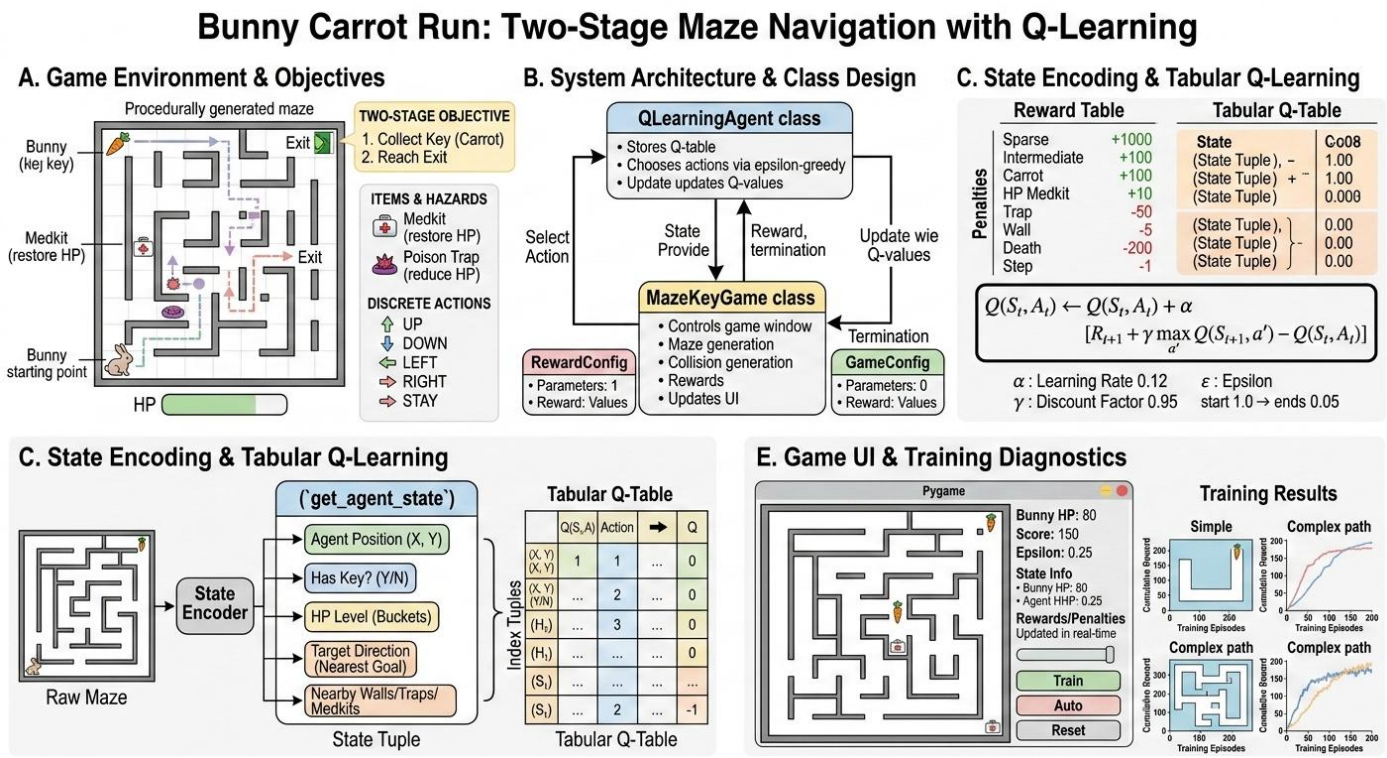

Fig 1. Workflow (This image was generated by Gemini based on my game flow.)

2.2 Class Design of the Game

MazeKeyGame class: The MazeKeyGame class controls the whole game. It creates the game window, generates the maze, and places the carrot, exit, traps, and medkits on the map. During the game, it updates the bunny's position, checks collisions, calculates rewards, and decides whether the episode ends. It also handles drawing the UI on the screen.

QLearningAgent class: The QLearningAgent class handles the learning part. It stores the Q-table and chooses actions using the epsilon-greedy method. After each step, it updates the Q-values using the Q-learning formula. Epsilon is gradually reduced after each episode. This class is responsible for learning a better policy by interacting with the game environment.

GameConfig class: The GameConfig class stores basic game settings, such as map size, maximum HP, number of traps and medkits, tile size, and frame rate.

Maze generation module: The method make sure the maze is valid and that the bunny can reach the key and exit before placing items on the map.

State encoding module (get_agent_state method): The get_agent_state method builds the state used by the Q-learning agent. It includes information such as the bunny's position, whether it has the key, HP level, direction to the target, nearby walls, and nearby traps.

2.3 UI Design

The UI is divided into two parts: the maze on the left and the control panel on the right. It also shows rewards and penalties in real time, so users can clearly see how the agent is learning during the game.

Fig 2. Bunny Carrot Maze UI

3 Implement of Q-learning Algorithm

In this project, we use the standard tabular Q-learning algorithm. Since the bunny moves in a grid maze and the actions are discrete (UP, DOWN, LEFT, RIGHT, STAY), we store Q-values in a Q-table instead of using a neural network.

The state is generated by the `get_agent_state()` function. During training, the agent chooses actions using the ε-greedy strategy. It sometimes explores randomly and sometimes selects the action with the highest Q-value.

After each step, the Q-value is updated using the Q-learning formula. The learning rate (alpha) is 0.12 and the discount factor (gamma) is 0.95. The epsilon value starts from 1.0 and gradually decreases to 0.05. Through many training episodes, the agent updates its Q-table and learns how to collect the carrot and reach the exit more effectively.

4 Experimental results

4.1 Experimental process

I conducted three experiments using three different maps. Since each map was generated separately, the layout of the walls, traps, carrot, and exit is different each time.

Fig 3. Map 1 Game Interface

Fig 4. Map 2 Game Interface

Fig 5. Map 3 Game Interface

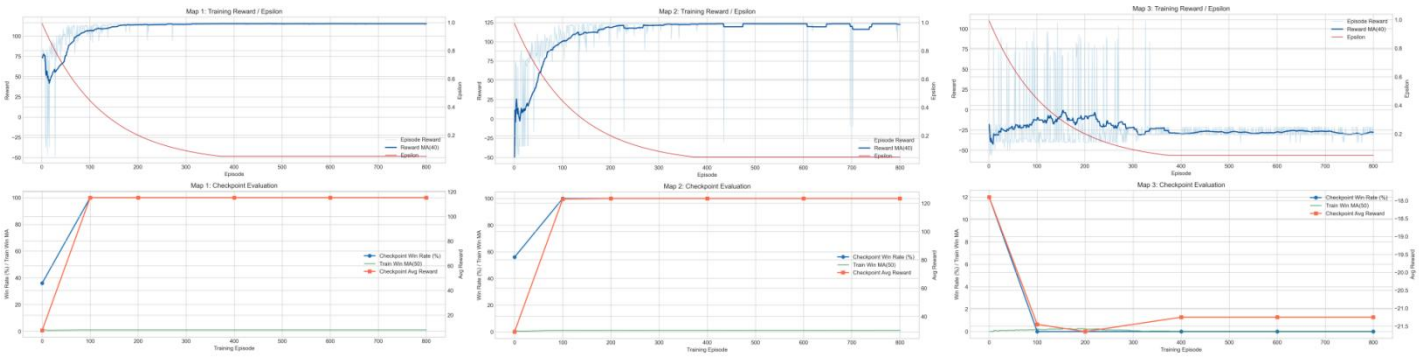

Fig 6. Training Logs (from left to right: Map1, 2, 3)

From the results, Map 1 and Map 2 can be learned successfully. In these two maps, the path from the start to the carrot and then to the exit is shorter and easier to follow. So during training, the agent can slowly figure out the correct route. However, Map 3 is more complicated. The path is longer, with more turns and possible loops. This makes it harder for the agent to plan ahead, and it often gets stuck or fails to reach the exit. This shows that Q-learning becomes less stable when the route is longer and the task requires more long-term planning.

4.2 Training Results

Map	Base Win (%)	Final Win (%)	Delta Win (pp)	Base Reward	Final Reward	Delta Reward	Base Steps	Final Steps	Delta Steps
Map 1	36.00	100.00	+64.00	7.62	115.35	+107.73	212.68	23.00	-189.68
Map 2	56.00	100.00	+44.00	29.21	123.40	+94.19	199.40	24.00	-175.40
Map 3	12.00	0.00	-12.00	-17.90	-21.25	-3.35	228.88	243.00	+14.12

Across the three maps, mean baseline win rate is 34.67%, and mean final win rate is 66.67%. This average gain masks a bimodal outcome: complete convergence on Map 1/2 and failure on Map 3.

Map	Early Win (%)	Mid Win (%)	Late Win (%)	Early Reward	Late Reward	First Ep (Roll50>=0.9)	Train Reward Std
Map 1	97.00	100.00	100.00	100.56	115.21	58	15.06
Map 2	91.50	99.50	98.50	92.67	120.92	69	27.10
Map 3	19.50	1.00	0.00	-14.18	-27.48	None	28.17

Map 1 and Map 2 enter stable high-performance regimes early (rolling-50 win rate >= 90% reached by episode 58 and 69, respectively). Map 3 never reaches this threshold and degrades from 19.5% early win rate to 0% in late training.

Overall, these two tables show a pattern: Q-learning works very well on Map 1 and Map 2, but it is not robust on Map 3. This also supports the idea that when the route is longer or the maze has more confusing structure, Q-learning becomes less stable and struggles with long-term planning.

5 Challenges and Solutions

During the experiment, one main challenge was designing a reward model that could truly guide the bunny to complete the two-stage task (collect carrot and reach exit). At first, I added more shaping rewards, such as distance-based rewards and exploration bonuses. However, I found that a more complex reward design did not always improve the learning result. In some cases, the agent focused too much on small intermediate

rewards instead of the final goal. This problem is also discussed in reward shaping research, which shows that improper reward design may change the learned policy in unexpected ways [2].

Another challenge appeared in Map 3. Although the Q-learning parameters were exactly the same as Map 1 and Map 2, the agent failed to learn a stable policy in Map 3. The main reason is that Map 3 has a longer and more complicated path from the start to the carrot and then to the exit. This makes the delayed reward problem more serious. In reinforcement learning, this is related to the credit assignment problem, where the final reward must be propagated back through many steps [3]. When the task horizon becomes longer, learning becomes slower and less stable. Previous studies also show that reinforcement learning algorithms can perform very differently across environments with different structures, even under the same hyperparameters [4].

In addition, I limited Q-value updates to feasible next actions to reduce noisy overestimation, which is related to ideas discussed in Double Q-learning [5]. Although this improved performance on Map 1 and Map 2, Map 3 still showed instability. This suggests that tabular Q-learning may not be robust enough when facing longer-horizon tasks and more complex maze topologies.

References

[1] Pygame: https://github.com/pygame/pygame

[2] Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. ICML.

[3] Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

[4] Henderson, P., et al. (2018). Deep reinforcement learning that matters. AAAI.

[5] Hasselt, H. v., Guez, A., & Silver, D. (2016). Deep reinforcement learning with Double Q-learning. AAAI.